Lista 7: Zaawansowane Grupowanie Danych

Uczenie Maszynowe – Uniwersytet Wrocławski

Semestr zimowy 2024/2025

Uwaga: Każde zadanie jest warte 2 punkty. Rozwiązania powinny zawierać kod w Pythonie oraz krótkie komentarze wyjaśniające kluczowe decyzje implementacyjne.

Zadanie 1: Semi-supervised Learning z Klasteryzacją (2 pkt)

Scenariusz: Firma "WroMed" gromadzi dane medyczne pacjentów, ale etykietowanie chorób przez lekarzy jest kosztowne i czasochłonne. Masz do dyspozycji duży zbiór nieetykietowanych danych i niewielki podzbiór (5%) z etykietami. Twoim zadaniem jest wykorzystanie klasteryzacji do propagacji etykiet na pozostałe dane.

Dane: Użyj zbioru `digits` ze `sklearn.datasets` (1797 obrazów cyfr 8x8, 10 klas).

Polecenia:

(a) **Symulacja braku etykiet:** Losowo ukryj 95% etykiet (oznacz jako -1). Zachowaj tylko 5% danych jako "etykietowane".

(b) **Klasteryzacja + propagacja:** Wykonaj klasteryzację K-Means ($k = 10$). Dla każdego klastra znajdź najczęstszą etykietę wśród etykietowanych punktów i przypisz ją wszystkim punktom w klastrze.

(c) **Porównanie metod:** Zaimplementuj dwie strategie propagacji:

- **Głosowanie:** Najczęstsza etykieta w klastrze (punkt b)
- **Najbliższy centroid:** Dla klastrów bez etykiet, przypisz etykietę najbliższego klastra z etykietami

(d) **Ewaluacja:** Oblicz accuracy na **nieetykietowanych** danych. Porównaj z:

- k-NN ($k = 5$) trenowanym tylko na etykietowanych danych
- Pełny nadzorowany klasyfikator (100% etykiet) – górna granica

Jak zmienia się accuracy w zależności od % dostępnych etykiet (1%, 5%, 10%, 20%)?

Zadanie 2: Anomalie w Sieci Czujników (2 pkt)

Scenariusz: Firma "WroExpress" monitoruje flotę 200 tramwajów za pomocą czujników temperatury, wibracji i prędkości. Po serii awarii, Twoje zadanie to wykrycie anomalnych zachowań tramwajów przed ich całkowitą awarią.

Dane: Skorzystaj z syntetycznych danych generowanych w kodzie startowym lub z własnych pomiarów symulujących zachowanie czujników.

Polecenia:

(a) **Metoda K-Means:** Zaimplementuj wykrywanie anomalii oparte na odległości od centroidów. Punkt jest anomalią, jeśli jego odległość od najbliższego centroidu przekracza próg (np. 95. percentyl).

(b) **Metoda DBSCAN:** Użyj punktów oznaczonych jako szum (label = -1) przez DBSCAN jako kandydatów na anomalie. Dobierz parametry `eps` i `min_samples`.

(c) **Metoda LOF:** Zastosuj Local Outlier Factor i porównaj wyniki z poprzednimi metodami.

(d) **Analiza porównawcza:** Dla każdej metody oblicz precision, recall i F1-score (zakładając, że znasz prawdziwe anomalie z danych testowych). Która metoda najlepiej wykrywa anomalie w tym scenariuszu? Uzasadnij.

Zadanie 3: Klasteryzacja Trajektorii GPS (2 pkt)

Scenariusz: Miasto Wrocław analizuje wzorce mobilności rowerzystów na podstawie danych z systemu "WroVelo". Trajektorie mają różne długości i prędkości, więc standardowa odległość euklidesowa nie działa.

Dane: Skorzystaj z syntetycznych trajektorii generowanych w kodzie startowym lub z publicznych zbiorów danych GPS (np. GeoLife, OpenStreetMap traces).

Polecenia:

(a) **Implementacja DTW:** Zaimplementuj algorytm Dynamic Time Warping do obliczania odległości między dwiema trajektoriami. Możesz użyć biblioteki `dtaidistance` lub napisać własną implementację.

(b) **Macierz odległości:** Oblicz macierz odległości DTW dla wszystkich par trajektorii. Zwróć uwagę na złożoność obliczeniową.

(c) **Hierarchiczne grupowanie (Ward):** Zastosuj metodę Warda z prekalkulowaną macierzą odległości. Zwizualizuj dendrogram i wybierz odpowiednią liczbę klastrów.

(d) **Interpretacja:** Dla każdego klastra narysuj reprezentatywne trajektorie. Jakie wzorce mobilności odkryłeś? (np. trasy do pracy, rekreacyjne, turystyczne)

Zadanie 4: WroShop - Przetwarzanie Danych na Dużej Skali (2 pkt)

Scenariusz: Firma "WroShop" prowadzi platformę e-commerce we Wrocławiu. Ich dane transakcyjne rosną wykładniczo i przekraczają możliwości pandas. Twoim zadaniem jest poznanie narzędzi do przetwarzania dużych danych: Dask i PySpark.

Dane:

- `data/wroshop_small.csv` – 100,000 transakcji (pojedynczy plik)
- `data/wroshop_medium/` – 1,000,000 transakcji (10 plików partycjonowanych)

Polecenia:

(a) **Pandas baseline (100K):** Wczytaj małe dane i wykonaj operacje: groupby po kategorii, top 10 miast, filtrowanie. Zmierz czas i użycie pamięci.

(b) **Pandas na większych danych (1M):** Wczytaj wszystkie pliki z `wroshop_medium/` używając `glob` i `pd.concat`. Powtórz operacje. Zaobserwuj spowolnienie.

(c) **Dask DataFrame:** Użyj `dask.dataframe`:

```
import dask.dataframe as dd
ddf = dd.read_csv('data/wroshop_medium/*.csv')
# Operacje sa LAZY - wykonaj .compute() na koncu
result = ddf.groupby('category')['total_amount'].mean().compute()
```

Zwizualizuj graf zadań. Porównaj czas z pandas.

(d) **PySpark DataFrame:** Użyj `pyspark` (lub Google Colab):

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("WroShop").getOrCreate()
df = spark.read.csv('data/wroshop_medium/', header=True,
    inferSchema=True)
df.groupBy('category').avg('total_amount').show()
```

Pokaż plan wykonania: `result.explain()`.

(e) **Porównanie:** Stwórz tabelę porównawczą czasów. Kiedy używać pandas/Dask/Spark?

Zadanie 5: WroAI - Mnożenie Macierzy w Praktyce ML (2 pkt)

Scenariusz: Startup "WroAI" buduje systemy uczenia maszynowego. Jako inżynier ML, Twoje zadanie to zoptymalizowanie kluczowych operacji obliczeniowych poprzez efektywne wykorzystanie mnożenia macierzy zamiast pętli.

Polecenia:

(a) **Softmax Attention:** Zaimplementuj mechanizm atencji z Transformerów:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Porównaj implementację z 3 zagnieżdżonymi pętlami vs wersję macierzową. Benchmark dla sekwencji długości 64, 256, 512.

(b) **Macierz pomyłek bez pętli:** Zaimplementuj obliczanie confusion matrix używając mnożenia macierzy: Benchmark dla 10K, 100K próbek i 10, 50, 100 klas.

(c) **Batch Normalization:** Zaimplementuj normalizację wsadową: Benchmark dla batch=128, features=1024, 4096.

(d) **PageRank:** Zaimplementuj algorytm PageRank jako iterację macierzową:

$$r_{t+1} = d \cdot M \cdot r_t + \frac{1-d}{n} \cdot \mathbf{1}$$

gdzie M to znormalizowana macierz przejścia. Porównaj z wersją z pętlami. Zwizualizuj zbieżność.

Wymagania: Dla każdej operacji: (1) implementacja naiwna, (2) wyprowadzenie macierzowe, (3) benchmark, (4) weryfikacja poprawności (`np.allclose`).

Wskazówki ogólne

- Zalecane biblioteki: `numpy`, `scipy`, `scikit-learn`, `matplotlib`, `networkx` (dla zadania 3), `dtaidistance` lub `tslearn` (dla zadania 4).
- Wykresy powinny mieć **tytuły, etykiety osi i legendy**.
- Kod powinien być **czytelny** i zawierać komentarze wyjaśniające kluczowe kroki.